UNIVERSITY PORTFOLIO PROJECT

DiScammer

A bilingual, privacy-conscious learning product for recognizing online scams.

Project author

Foma Molchanov

DiScammer combines an accessible reading experience with hands-on checking tools. Its product goal is practical: help a person pause before acting on a suspicious message, link, or request.

2	9	55	23
authored languages	course modules	lesson pairs	React islands

What the product contains

A Russian and English website with a journal archive, ten scam-type guides, a structured course, a local message checker, a local link inspector, and a separately served Unity WebGL learning game.

Public portfolio dossier: /en/portfolio and /ru/portfolio. The dossier separates implemented facts from the contribution and tooling account Foma must review before submission.

Honesty is a product requirement

The project does not present a browser-local completion record as a certificate, a verified credential, or proof of competence. It does not claim measured learning outcomes without a real study.

Product intent and boundaries

DiScammer is designed for Russian- and English-speaking people who want to understand modern fraud patterns without being treated as security professionals. It aims to make a useful hesitation routine visible and repeatable.

Implemented product surface

Surface	What is implemented
Guides	Ten authored EN/RU Markdown pairs with a static catalogue of 12 linked FTC, FBI, CISA, and Investor.gov primary-source entries. The sources support general guidance, not live alerts.
Journal	A finite archive of interviews, newsletters, and articles - not a demonstrated subscription or weekly publishing service.
Course	Nine modules and 55 EN/RU lesson pairs, combining reading, quizzes, and scenario practice.
Check tools	Message and link analysis performed in the browser, with no raw checker input sent to an analysis API.
Learning record	Browser-local progress, self-assessment result, and printable personal record. It can disappear if browser storage is cleared.
Game	The independent Unity WebGL build Antifraud Hero, served under /play/.

Trust boundaries

The final self-assessment is self-paced, untimed, unproctored, and unmonitored.

A local record is not a verified credential, identity check, institutional issuance, or external transcript.

Local analysis does not erase normal operational access logs or privacy-scrubbed, sampled client-error reporting.

Why this framing matters

An anti-scam product should show the same care with its own claims that it asks users to apply to suspicious messages. The repository's claim ledger is therefore a release gate, not a marketing checklist.

Technical architecture

The site uses FastAPI and Jinja2 to send complete, localized HTML first. React enhances only interactive regions, so reading and navigation remain grounded in server-rendered pages.

Browser request

FastAPI router -> Jinja page -> optional React island -> content or local browser state

SEO router -> robots, sitemap, RSS, and web manifest | Game router -> Unity WebGL

Key engineering decisions

Surface	What is implemented
Server rendering	FastAPI/Jinja2 renders the multilingual reading surfaces and compatibility redirects; page behavior is defined by the pages, API, game, and SEO routers.
React islands	23 lazy island registrations hydrate marked elements only. A captured server HTML fallback remains visible while a chunk loads or if an island fails.
Asset delivery	Vite writes content-hashed JS and CSS plus a manifest. The backend reads the manifest instead of depending on stable app.js or main.css filenames.
Content pipeline	Markdown sources feed the journal and guides. A checked-in provenance catalogue maps every guide to one to three dated, bilingual primary-source entries. Course module metadata is declared in code; lesson bodies and widget data are JSON files with paired locales.
Runtime store	SQLite keeps derived post, guide, module, lesson, and search data. Startup reseeding is file-locked and deliberately excludes durable user state.

Design principle

The game is the visual home for deeper cyberpunk immersion. The website is a calmer Field Office: specific, technically grounded, and readable on mobile in both Cyrillic and Latin scripts.

Quality, privacy, and review

DiScammer's repository checks engineering behavior. They are valuable evidence of care, but they are not evidence that users learned more or that an external organisation certified the product.

Repository quality gates

Surface	What is implemented
Backend	21 pytest modules cover routes, content lifecycle, security headers, search, localization, health, game handling, observability behavior, the Product Truth Gate, and guide provenance.
Frontend	5 Vitest files validate client-side utilities and interactive logic; the production build type-checks TypeScript and regenerates committed assets.
End-to-end	21 Playwright specifications exercise the composed product in CI, including the principal user flows and privacy-oriented checks.
Localization	The i18n audit verifies string use, missing/dead keys, and EN/RU key parity across server templates, Python, and React islands.
Delivery	CI runs dependency audits, backend tests, frontend tests/build, static-asset synchronization, Docker build, and E2E verification.

Claim governance

docs/portfolio/CLAIM_LEDGER.md tracks every changed, removed, in-progress, and unresolved public claim.

A public claim must point to an implemented capability or a named primary source. A local browser value, internal review, or visual code never becomes external verification by wording alone.

What remains intentionally unclaimed

There is no user-outcome study, no institutional endorsement, no real-time threat-intelligence claim, no account-backed learning transcript, and no verified credential. Static source provenance now supports the guides; any impact claim still requires consented research with appropriately limited findings.

Portfolio review note

A reviewer should be able to distinguish Foma's own decisions and work from mentoring, libraries, design assets, and AI-assisted tools. Those details require Foma's review; the repository deliberately does not invent them.

Evidence map and next step

The project is strongest when its product story, technical implementation, and evidence limits agree. The following source order keeps those layers aligned.

Authority order

Surface	What is implemented
1. Executable evidence	Code, schemas, tests, generated Vite manifest, and named local or deployed checks.
2. Product intent	PRODUCT.md and DESIGN.md define the current user need, language, interaction principles, and Field Office visual system.
3. Public truth	docs/portfolio/CLAIM_LEDGER.md defines the allowable wording and the claims still needing evidence or removal.
4. Future work	FEATURE_PLAN.md is a backlog. Its status is not proof of configuration, staging, or production operation.
5. Historical context	Earlier redesign plans, handoffs, old screenshots, and earlier PDFs explain decisions but do not override current evidence.

Status vocabulary

Implemented: source and repository checks exist. Locally verified: a named command or visual check passed on a stated revision.

Configured: a configuration or runbook exists. Staging verified and production verified require dated checks against those environments.

Recommended continuation

Keep the source catalogue current only when its links and scope can be checked, and have Foma replace or approve the provisional first-person dossier before any application. Then test an incident-response flow with real, consented participants before making impact claims. That sequence strengthens both the product and the university portfolio without manufacturing evidence.

Prepared from the current repository overview in PROJECT.md.